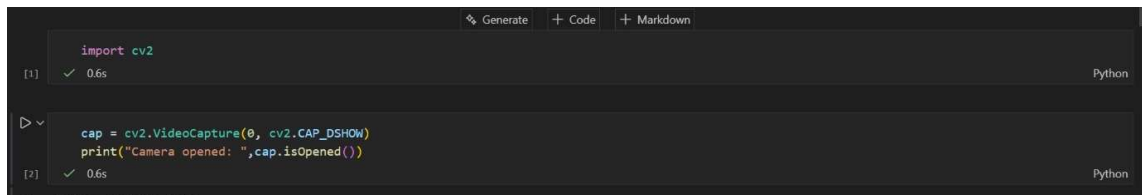


On Device Image Recognition

1. Importing Libraries and Checking live video from laptop camera

This initial script checks if OpenCV can successfully communicate with your default laptop camera.

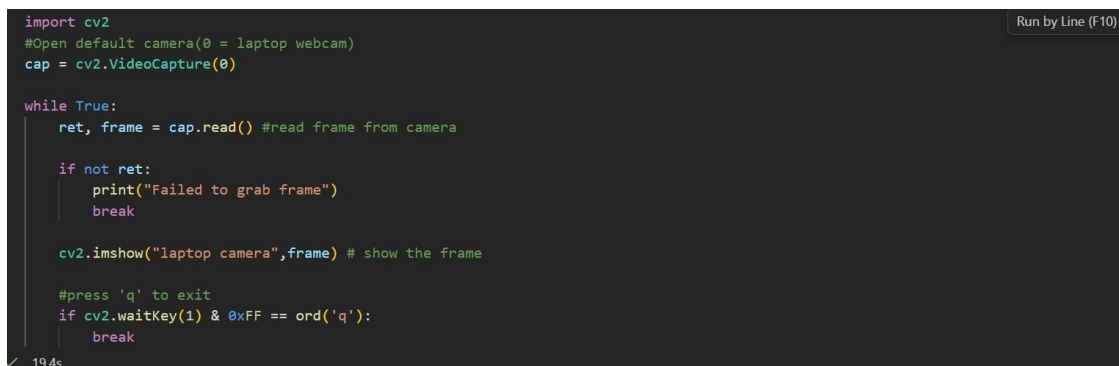


```
import cv2

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
print("Camera opened: ",cap.isOpened())
```

2. Capturing video from Laptop camera

This sets up a continuous loop to grab frames from your webcam and display them in a window



```
import cv2
#Open default camera(0 = laptop webcam)
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read() #read frame from camera

    if not ret:
        print("Failed to grab frame")
        break

    cv2.imshow("laptop camera",frame) # show the frame

    #press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

3. Loading sample video from local device

Instead of a live feed, this script reads and plays an offline video file.

```
import cv2

cap = cv2.VideoCapture("sample_video.mp4")

while cap.isOpened():
    ret, frame = cap.read()

    if not ret:
        break

    cv2.imshow("Video", frame)

    if cv2.waitKey(25) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

✓ 7.4s

4. Analyzing the sample offline video's Grayscale ,RGB,Negative,

Reads the first frame of a video, converts it into three different color spaces, plots them using Matplotlib, and prints the exact BGR values of the pixels.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load video
cap = cv2.VideoCapture("sample_video.mp4") # change to your video path

ret, frame = cap.read()

if not ret:
    print("Error reading video")
    cap.release()
    exit()

# Convert formats
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
negative = 255 - frame

# Display images in output cell
plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.title("Original")
plt.imshow(rgb)
```

```
plt.subplot(2,2,2)
plt.title("Gray")
plt.imshow(gray, cmap="gray")
plt.axis("off")

plt.subplot(2,2,3)
plt.title("RGB")
plt.imshow(rgb)
plt.axis("off")

plt.subplot(2,2,4)
plt.title("Negative")
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()

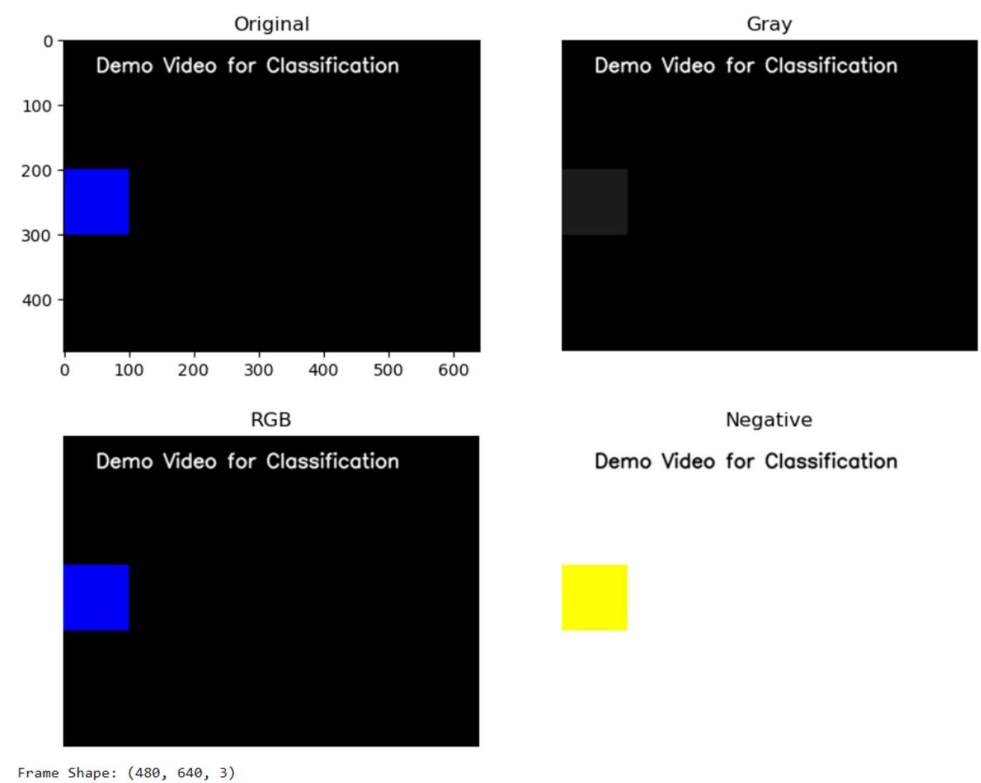
# Print pixel values of first frame
h, w, c = frame.shape

print("Frame Shape:", frame.shape)
print("\nPixel values of first frame:\n")

for i in range(h):
    for j in range(w):
        print(f"Pixel ({i},{j}) BGR:", frame[i,j])
```

✓ 25.2s

Output:



Showing pixel values:

Pixel (1072,28)	BGR: [18 20 13]
Pixel (1072,29)	BGR: [19 21 14]
Pixel (1072,30)	BGR: [19 21 14]
Pixel (1072,31)	BGR: [18 20 13]
Pixel (1072,32)	BGR: [19 18 12]
Pixel (1072,33)	BGR: [19 18 12]
Pixel (1072,34)	BGR: [19 18 12]
Pixel (1072,35)	BGR: [19 18 12]
Pixel (1072,36)	BGR: [20 19 13]
Pixel (1072,37)	BGR: [20 19 13]
Pixel (1072,38)	BGR: [20 19 13]
Pixel (1072,39)	BGR: [20 19 13]
Pixel (1072,40)	BGR: [19 18 12]
Pixel (1072,41)	BGR: [19 18 12]
Pixel (1072,42)	BGR: [19 18 12]
Pixel (1072,43)	BGR: [19 18 12]
Pixel (1072,44)	BGR: [20 19 13]
Pixel (1072,45)	BGR: [20 19 13]

5. Loading Offline sample video and printing total frames of a video,fps,width and height of the frame.

This uses `cap.get()` to extract and display the video's metadata, such as frame count, frames per second (FPS), and resolution.

```
#Display Video Properties

import cv2

cap = cv2.VideoCapture("sample_video.mp4")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap.get(cv2.CAP_PROP_FPS)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print("Total frames:",total_frames)
print("fps:",fps)
print("width",width)
print("height:",height)

cap.release()
✓ 0.0s
```

```
Total frames: 241
fps: 24.0
width 612.0
height: 1080.0
```

6. Loading video and converting into Grayscale video:

This streams an offline video but converts every frame to grayscale before displaying it.

```

import cv2

cap = cv2.VideoCapture("sample_video.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Grayscale video:", gray)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

✓ 2.9s

7. Creating Rectangle and circle using opencv

This opens the webcam in a fullscreen window and continuously overlays a drawn rectangle, circle, and text onto the live feed .

```

import cv2

cap = cv2.VideoCapture(0)

# Create fullscreen window
cv2.namedWindow("Shapes", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Shapes", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Draw rectangle
    cv2.rectangle(frame, (100,100), (300,300), (0,255,0), 3)

    # Draw circle
    cv2.circle(frame, (200,200), 50, (255,0,0), 3)

    # Put text
    cv2.putText(frame, "OpenCV practice", (50,50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

    cv2.imshow("Shapes", frame)

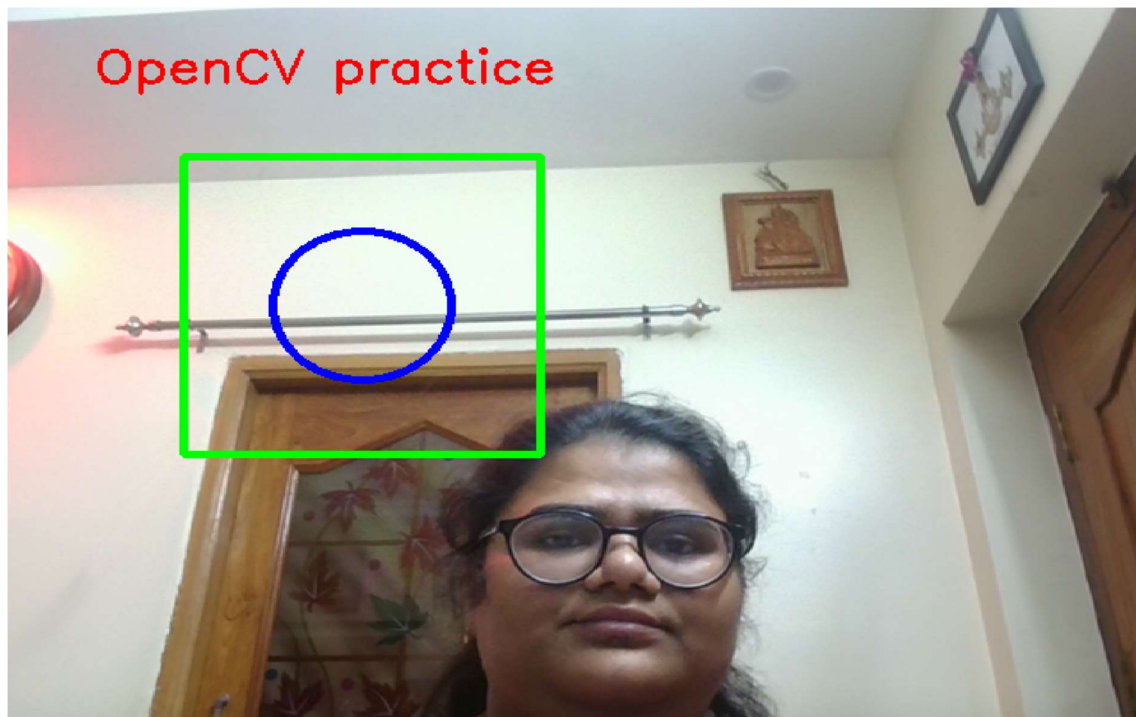
```

```
cv2.imshow('Shapes', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
✓ 15.3s
```

Output:



8. Converting live video to find edge using canny operator:

This part applies the Canny edge detection algorithm to a live camera feed

Mathematical Equation:

Step 1: Convert Image to Grayscale

$$Gray = 0.299R + 0.587G + 0.114B$$

Step 2: Noise Reduction (Gaussian Blur)

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

The image is convolved with the Gaussian kernel:

$$I_s(x, y) = I(x, y) * G(x, y)$$

Step 3: Compute Image Gradient it finds G_x and G_y

Step 4: Gradient Magnitude (Edge Strength)

$$G = |G_x| + |G_y|$$

Step 5: Gradient Direction (Edge Orientation)

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

$$G(x, y) > T_H$$

$$T_L < G(x, y) < T_H$$

$$G(x, y) < T_L$$

Step 6: Non-Maximum Suppression

Steps of Canny Edge Detection:

1. **Convert the image to grayscale** - reduce the image from RGB to a single intensity channel.
2. **Apply Gaussian blur** - smooth the image to remove noise.
3. **Calculate image gradients** - find how quickly the intensity changes in horizontal and vertical directions.

4. **Compute gradient magnitude and direction** - determine the strength and orientation of edges.
5. **Apply non-maximum suppression** - thin the edges by keeping only the strongest pixels.
6. **Apply double thresholding** - classify pixels as strong edges, weak edges, or non-edges.
7. **Perform edge tracking by hysteresis** - keep weak edges that are connected to strong edges and remove the rest.

```
import cv2

# Open webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Cannot open camera")
    exit()

# Create fullscreen windows
cv2.namedWindow("Original Video", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Original Video", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

cv2.namedWindow("Canny Edge Detection", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Canny Edge Detection", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    # Capture frame
    ret, frame = cap.read()

    if not ret:
        print("Can't receive frame")
        break
```

```
if not ret:
    print("Can't receive frame")
    break

# Convert to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Gaussian blur
blur = cv2.GaussianBlur(gray, (5,5), 0)

# Canny edge detection
edges = cv2.Canny(blur, 100, 200)

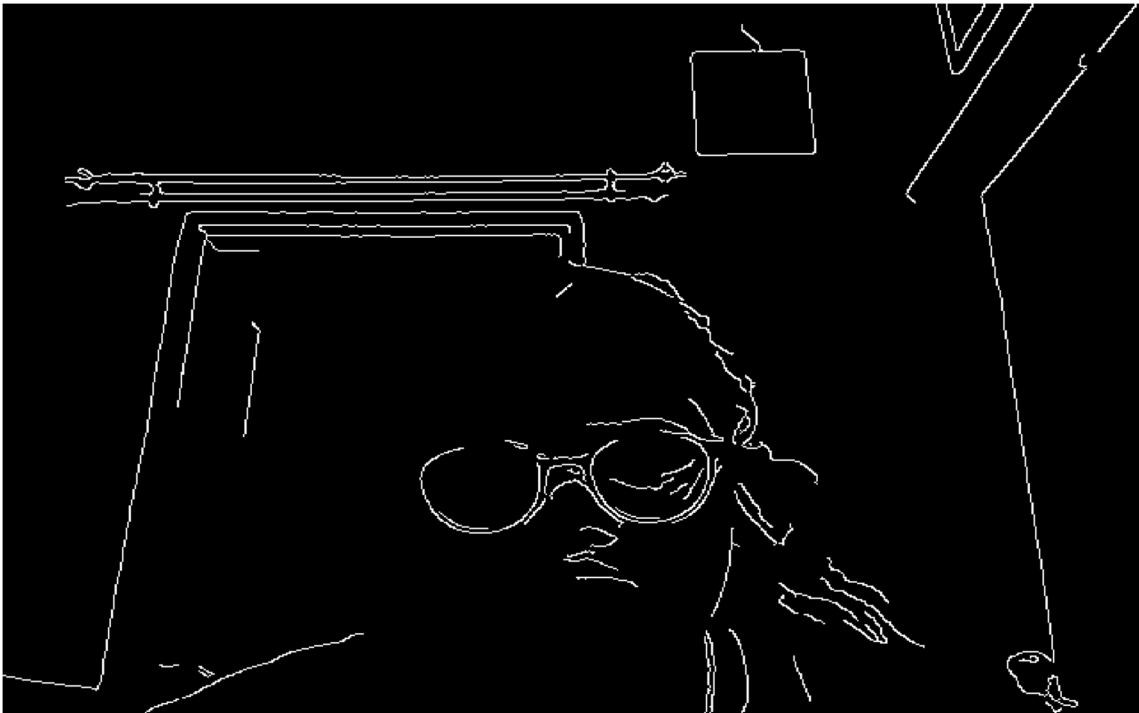
# Display frames
cv2.imshow("Original Video", frame)
cv2.imshow("Canny Edge Detection", edges)

# Press q to exit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

✓ 11.3s
```

Output:



9. Converting Live video to Negative video:

This creates a real-time negative visual effect by subtracting the current pixel intensity from the maximum possible intensity.

```
import cv2

# Open webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Cannot open camera")
    exit()

# Create fullscreen window
cv2.namedWindow("Negative Video", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Negative Video", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    ret, frame = cap.read()

    if not ret:
        print("Can't receive frame")
        break

    # Create negative image
    negative = 255 - frame
```

```
    # Show negative video
    cv2.imshow("Negative Video", negative)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    cap.release()
    cv2.destroyAllWindows()

✓ 23.6s
```

Output:



Mathematical Equation

For an 8-bit image:

$$s = 255 - r$$

Where:

- r = original pixel value
- s = negative pixel value
- 255 = maximum intensity value in an 8-bit image

10. Detecting the motion if motion found edge will be shown otherwise black screen will be shown.

This compares consecutive frames to detect motion. If the amount of motion exceeds a threshold, it displays the edges; otherwise, the screen remains black

Mathematical Equation:

1. Grayscale Conversion

$$Gray(x, y) = 0.299R(x, y) + 0.587G(x, y) + 0.114B(x, y)$$

2. Frame Difference (Motion Detection):

$$D(x, y) = |G_t(x, y) - G_{t-1}(x, y)|$$

```
import cv2
import numpy as np

# Open webcam
cap = cv2.VideoCapture(0)

# Fullscreen window
cv2.namedWindow("Motion Edge", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Motion Edge", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

# Read first frame
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert current frame to gray
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

# Frame difference
diff = cv2.absdiff(prev_gray, gray)

# Threshold to detect motion
_, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

motion_pixels = cv2.countNonZero(thresh)

# If motion detected
if motion_pixels > 5000:

    # Apply Canny Edge
    edges = cv2.Canny(gray, 100, 200)

    cv2.imshow("Motion Edge", edges)

else:

    # Show black screen
    black = np.zeros_like(gray)
    cv2.imshow("Motion Edge", black)

# Update previous frame
prev_gray = gray

# Quit key
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

```

cap.release()
cv2.destroyAllWindows()

```

✓ 1m 19.0s

Output:

https://drive.google.com/file/d/1txPD6iVlzqqqKN1MvwNhL2ElmyUG3mT7/view?usp=drive_link